

How many operations does it take to evaluate the sum (6.7.12)? Direct evaluation of $l_{\max}^2$ terms at N sample points is an $O(N^2)$ process. You might try to speed this up by choosing the sample points ϕ_j to be equally spaced in angle and doing the sum over m by an FFT. Each FFT is $O(N_\phi \ln N_\phi)$, and you have to do $O(N_\theta l_{\max})$ of them, for a total of $O(N^{3/2} \ln N)$ operations, which is some improvement. A simple rearrangement [3-5] gives an even better way: Interchange the order of summation

$$\sum_{l=0}^{l_{\max}} \sum_{m=-l}^l \longleftrightarrow \sum_{m=-l_{\max}}^{l_{\max}} \sum_{l=|m|}^{l_{\max}} \quad (6.7.13)$$

so that

$$f(\theta_i, \phi_j) = \sum_{m=-l_{\max}}^{l_{\max}} q_m(\theta_i) e^{im\phi_j} \quad (6.7.14)$$

where

$$q_m(\theta_i) = \sum_{l=|m|}^{l_{\max}} a_{lm} \tilde{P}_l^m(\cos \theta_i) \quad (6.7.15)$$

Evaluating the sum in (6.7.15) is $O(l_{\max})$, and one must do this for $O(l_{\max} N_\theta)$ q_m 's, so the total work is $O(N^{3/2})$. To evaluate equation (6.7.14) by an FFT at fixed θ_i is $O(N_\phi \ln N_\phi)$. There are N_θ FFTs to be done, for a total operations count of $O(N \ln N)$, which is negligible in comparison. So the total algorithm is $O(N^{3/2})$. Note that you can evaluate equation (6.7.14) either by precomputing and storing the $\tilde{P}_l^m$'s using the recurrence relation (6.7.9), or by Clenshaw's method (§5.4).

What about inverting the transform (6.7.12)? The formal inverse for the expansion of a continuous function $f(\theta, \phi)$ follows from the orthonormality of the Y_{lm} 's, equation (6.7.1),

$$a_{lm} = \int \sin \theta \, d\theta \, d\phi \, f(\theta, \phi) e^{-im\phi} \tilde{P}_l^m(\cos \theta) \quad (6.7.16)$$

For the discrete case, where we have a sampled function, the integral becomes a quadrature:

$$a_{lm} = \sum_{i,j} w(\theta_i) f(\theta_i, \phi_j) e^{-im\phi_j} \tilde{P}_l^m(\cos \theta_i) \quad (6.7.17)$$

Here $w(\theta_i)$ are the quadrature weights. In principle we could consider weights that depend on ϕ_j as well, but in practice we do the ϕ quadrature by an FFT, so the weights are unity. A good choice for the weights for an equi-angular grid in θ is given in Ref. [3], Theorem 3. Another possibility is to use Gaussian quadrature for the θ integral. In this case, you choose the sample points so that the $\cos \theta_i$'s are the abscissas returned by `gauleg` and the $w(\theta_i)$'s are the corresponding weights. The best way to organize the calculation is to first do the FFTs, computing

$$g_m(\theta_i) = \sum_j f(\theta_i, \phi_j) e^{-im\phi_j} \quad (6.7.18)$$

Then

$$a_{lm} = \sum_i w(\theta_i) g_m(\theta_i) \tilde{P}_l^m(\cos \theta_i) \quad (6.7.19)$$

You can verify that the operations count is dominated by equation (6.7.19) and scales as $O(N^{3/2})$ once again. In a real calculation, you should exploit all the symmetries that let you reduce the workload, such as $g_{-m} = g_m^*$ and $\tilde{P}_l^m[\cos(\pi - \theta)] = (-1)^{l+m} \tilde{P}_l^m(\cos \theta)$.

Very recently, algorithms for fast Legendre transforms have been developed, similar in spirit to the FFT [3,6,7]. Theoretically, they reduce the forward and inverse spherical harmonic transforms to $O(N \log^2 N)$ problems. However, current implementations [8] are not much faster than the $O(N^{3/2})$ methods above for $N \sim 500$, and there are stability and accuracy problems that require careful attention [9]. Stay tuned!

CITED REFERENCES AND FURTHER READING:

- Magnus, W., and Oberhettinger, F. 1949, *Formulas and Theorems for the Functions of Mathematical Physics* (New York: Chelsea), pp. 54ff.[1]
- Abramowitz, M., and Stegun, I.A. 1964, *Handbook of Mathematical Functions* (Washington: National Bureau of Standards); reprinted 1968 (New York: Dover); online at <http://www.nist.gov/aands>, Chapter 8.[2]
- Driscoll, J.R., and Healy, D.M. 1994, "Computing Fourier Transforms and Convolutions on the 2-sphere," *Advances in Applied Mathematics*, vol. 15, pp. 202–250.[3]
- Muciaccia, P.F., Natoli, P., and Vittorio, N. 1997, "Fast Spherical Harmonic Analysis: A Quick Algorithm for Generating and/or Inverting Full-Sky, High-Resolution Cosmic Microwave Background Anisotropy Maps," *Astrophysical Journal*, vol. 488, pp. L63–66.[4]
- Oh, S.P., Spergel, D.N., and Hinshaw, G. 1999, "An Efficient Technique to Determine the Power Spectrum from Cosmic Microwave Background Sky Maps," *Astrophysical Journal*, vol. 510, pp. 551–563, Appendix A.[5]
- Healy, D.M., Rockmore, D., Kostelec, P.J., and Moore, S. 2003, "FFTs for the 2-Sphere: Improvements and Variations," *Journal of Fourier Analysis and Applications*, vol. 9, pp. 341–385.[6]
- Potts, D., Steidl, G., and Tasche, M. 1998, "Fast and Stable Algorithms for Discrete Spherical Fourier Transforms," *Linear Algebra and Its Applications*, vol. 275–276, pp. 433–450.[7]
- Moore, S., Healy, D.M., Rockmore, D., and Kostelec, P.J. 2007+, *SpharmonicKit*. Software at <http://www.cs.dartmouth.edu/~geelong/sphere>. [8]
- Healy, D.M., Kostelec, P.J., and Rockmore, D. 2004, "Towards Safe and Effective High-Order Legendre Transforms with Applications to FFTs for the 2-Sphere," *Advances in Computational Mathematics*, vol. 21, pp. 59–105.[9]

6.8 Fresnel Integrals, Cosine and Sine Integrals

6.8.1 Fresnel Integrals

The two Fresnel integrals are defined by

$$C(x) = \int_0^x \cos\left(\frac{\pi}{2}t^2\right) dt, \quad S(x) = \int_0^x \sin\left(\frac{\pi}{2}t^2\right) dt \quad (6.8.1)$$

and are plotted in Figure 6.8.1.

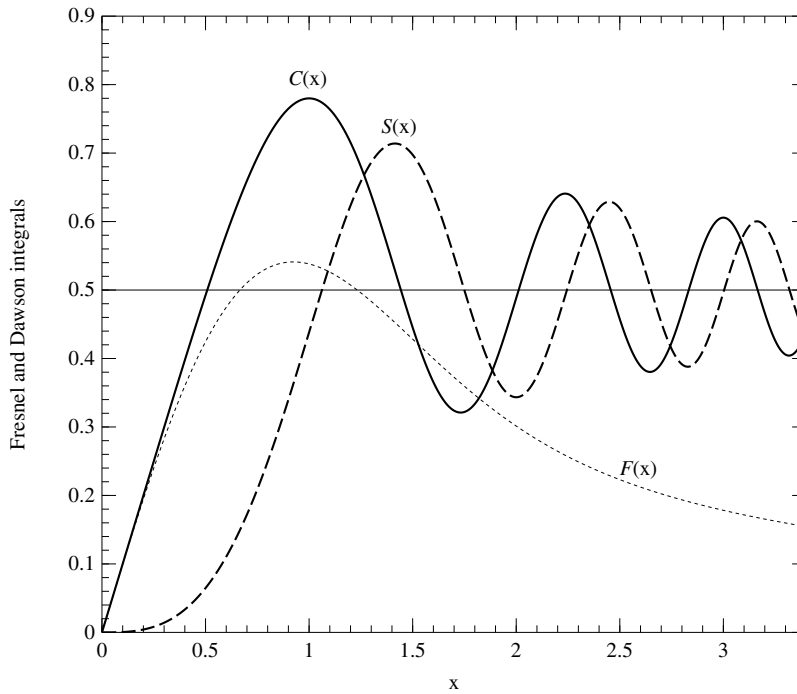


Figure 6.8.1. Fresnel integrals $C(x)$ and $S(x)$ (§6.8), and Dawson's integral $F(x)$ (§6.9).

The most convenient way of evaluating these functions to arbitrary precision is to use power series for small x and a continued fraction for large x . The series are

$$\begin{aligned} C(x) &= x - \left(\frac{\pi}{2}\right)^2 \frac{x^5}{5 \cdot 2!} + \left(\frac{\pi}{2}\right)^4 \frac{x^9}{9 \cdot 4!} - \cdots \\ S(x) &= \left(\frac{\pi}{2}\right) \frac{x^3}{3 \cdot 1!} - \left(\frac{\pi}{2}\right)^3 \frac{x^7}{7 \cdot 3!} + \left(\frac{\pi}{2}\right)^5 \frac{x^{11}}{11 \cdot 5!} - \cdots \end{aligned} \quad (6.8.2)$$

There is a complex continued fraction that yields both $S(x)$ and $C(x)$ simultaneously:

$$C(x) + iS(x) = \frac{1+i}{2} \operatorname{erf} z, \quad z = \frac{\sqrt{\pi}}{2}(1-i)x \quad (6.8.3)$$

where

$$\begin{aligned} e^{z^2} \operatorname{erfc} z &= \frac{1}{\sqrt{\pi}} \left(\frac{1}{z +} \frac{1/2}{z +} \frac{1}{z +} \frac{3/2}{z +} \frac{2}{z +} \cdots \right) \\ &= \frac{2z}{\sqrt{\pi}} \left(\frac{1}{2z^2 + 1 -} \frac{1 \cdot 2}{2z^2 + 5 -} \frac{3 \cdot 4}{2z^2 + 9 -} \cdots \right) \end{aligned} \quad (6.8.4)$$

In the last line we have converted the “standard” form of the continued fraction to its “even” form (see §5.2), which converges twice as fast. We must be careful not to evaluate the alternating series (6.8.2) at too large a value of x ; inspection of the terms shows that $x = 1.5$ is a good point to switch over to the continued fraction.

Note that for large x

$$C(x) \sim \frac{1}{2} + \frac{1}{\pi x} \sin\left(\frac{\pi}{2}x^2\right), \quad S(x) \sim \frac{1}{2} - \frac{1}{\pi x} \cos\left(\frac{\pi}{2}x^2\right) \quad (6.8.5)$$

Thus the precision of the routine `frenel` may be limited by the precision of the library routines for sine and cosine for large x .

Complex `frenel(const Doub x) {`

`frenel.h`

Computes the Fresnel integrals $S(x)$ and $C(x)$ for all real x . $C(x)$ is returned as the real part of `cs` and $S(x)$ as the imaginary part.

```
static const Int MAXIT=100;
static const Doub PI=3.141592653589793238, PIBY2=(PI/2.0), XMIN=1.5,
    EPS=numeric_limits<Doub>::epsilon(),
    FPMIN=numeric_limits<Doub>::min(),
    BIG=numeric_limits<Doub>::max()*EPS;
Here MAXIT is the maximum number of iterations allowed; EPS is the relative error;
FPMIN is a number near the smallest representable floating-point number; BIG is a
number near the machine overflow limit; and XMIN is the dividing line between using
the series and continued fraction.

Bool odd;
Int k,n;
Doub a,ax,fact,pix2,sign,sum,sumc,sums,term,test;
Complex b,cc,d,h,del,cs;
if ((ax=abs(x)) < sqrt(FPMIN)) {
    cs=ax;
    } else if (ax <= XMIN) {
        sum=sums=0.0;
        sumc=ax;
        sign=1.0;
        fact=PIBY2*ax*ax;
        odd=true;
        term=ax;
        n=3;
        for (k=1;k<=MAXIT;k++) {
            term *= fact/k;
            sum += sign*term/n;
            test=abs(sum)*EPS;
            if (odd) {
                sign = -sign;
                sums=sum;
                sum=sumc;
            } else {
                sumc=sum;
                sum=sums;
            }
            if (term < test) break;
            odd=!odd;
            n += 2;
        }
        if (k > MAXIT) throw("series failed in frenel");
        cs=Complex(sumc,sums);
    } else {
        pix2=PI*ax*ax;
        b=Complex(1.0,-pix2);
        cc=BIG;
        d=h=1.0/b;
        n = -1;
        for (k=2;k<=MAXIT;k++) {
            n += 2;
            a = -n*(n+1);
            b += 4.0;
            d=1.0/(a*d+b);
            cc=b+a/cc;
            del=cc*d;
            h *= del;
            if (abs(real(del)-1.0)+abs(imag(del)) <= EPS) break;
        }
    }
```

Special case: Avoid failure of convergence
test because of underflow.

Evaluate both series simultaneously.

Evaluate continued fraction by modified
Lentz's method (§5.2).

Denominators cannot be zero.

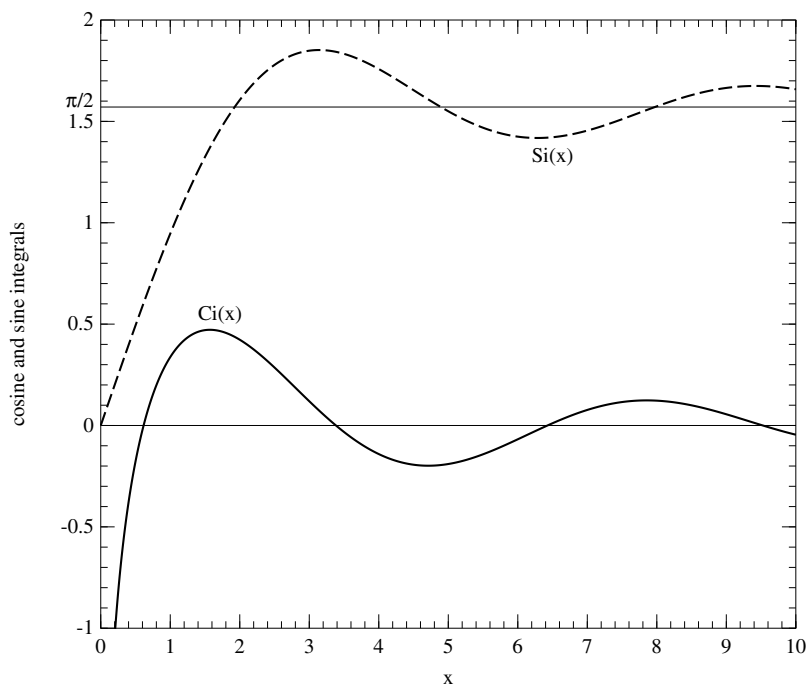


Figure 6.8.2. Sine and cosine integrals $\text{Si}(x)$ and $\text{Ci}(x)$.

```

if (k > MAXIT) throw("cf failed in frenel");
h *= Complex(ax,-ax);
cs=Complex(0.5,0.5)
    *(1.0-Complex(cos(0.5*pix2),sin(0.5*pix2))*h);
}
if (x < 0.0) cs = -cs;           Use antisymmetry.
return cs;
}

```

6.8.2 Cosine and Sine Integrals

The cosine and sine integrals are defined by

$$\begin{aligned}\text{Ci}(x) &= \gamma + \ln x + \int_0^x \frac{\cos t - 1}{t} dt \\ \text{Si}(x) &= \int_0^x \frac{\sin t}{t} dt\end{aligned}\tag{6.8.6}$$

and are plotted in Figure 6.8.2. Here $\gamma \approx 0.5772\dots$ is Euler's constant. We only need a way to calculate the functions for $x > 0$, because

$$\text{Si}(-x) = -\text{Si}(x), \quad \text{Ci}(-x) = \text{Ci}(x) - i\pi\tag{6.8.7}$$

Once again we can evaluate these functions by a judicious combination of power


```

if (t < sqrt(FPMIN)) {
    sumc=0.0;
    sums=t;
} else {
    sum=sums=sumc=0.0;
    sign=fact=1.0;
    odd=true;
    for (k=1;k<=MAXIT;k++) {
        fact *= t/k;
        term=fact/k;
        sum += sign*term;
        err=term/abs(sum);
        if (odd) {
            sign = -sign;
            sums=sum;
            sum=sumc;
        } else {
            sumc=sum;
            sum=sums;
        }
        if (err < EPS) break;
        odd=!odd;
    }
    if (k > MAXIT) throw("maxits exceeded in cisi");
}
cs=Complex(sumc+log(t)+EULER,sums);
}
if (x < 0.0) cs = conj(cs);
return cs;
}

```

Special case: Avoid failure of convergence test because of underflow.

CITED REFERENCES AND FURTHER READING:

- Stegun, I.A., and Zucker, R. 1976, "Automatic Computing Methods for Special Functions. III. The Sine, Cosine, Exponential integrals, and Related Functions," *Journal of Research of the National Bureau of Standards*, vol. 80B, pp. 291–311; 1981, "Automatic Computing Methods for Special Functions. IV. Complex Error Function, Fresnel Integrals, and Other Related Functions," *op. cit.*, vol. 86, pp. 661–686.
- Abramowitz, M., and Stegun, I.A. 1964, *Handbook of Mathematical Functions* (Washington: National Bureau of Standards); reprinted 1968 (New York: Dover); online at <http://www.nist.gov/aands>, Chapters 5 and 7.

6.9 Dawson's Integral

Dawson's Integral $F(x)$ is defined by

$$F(x) = e^{-x^2} \int_0^x e^{t^2} dt \quad (6.9.1)$$

See Figure 6.8.1 for a graph of the function. The function can also be related to the complex error function by

$$F(z) = \frac{i\sqrt{\pi}}{2} e^{-z^2} [1 - \operatorname{erfc}(-iz)]. \quad (6.9.2)$$

A remarkable approximation for $F(z)$, due to Rybicki [1], is

$$F(z) = \lim_{h \rightarrow 0} \frac{1}{\sqrt{\pi}} \sum_{n \text{ odd}} \frac{e^{-(z-nh)^2}}{n} \quad (6.9.3)$$

What makes equation (6.9.3) unusual is that its accuracy increases *exponentially* as h gets small, so that quite moderate values of h (and correspondingly quite rapid convergence of the series) give very accurate approximations.

We will discuss the theory that leads to equation (6.9.3) later, in §13.11, as an interesting application of Fourier methods. Here we simply implement a routine for real values of x based on the formula.

It is first convenient to shift the summation index to center it approximately on the maximum of the exponential term. Define n_0 to be the even integer nearest to x/h , and $x_0 \equiv n_0 h$, $x' \equiv x - x_0$, and $n' \equiv n - n_0$, so that

$$F(x) \approx \frac{1}{\sqrt{\pi}} \sum_{n'=-N}^N \frac{e^{-(x'-n'h)^2}}{n' + n_0} \quad (6.9.4)$$

where the approximate equality is accurate when h is sufficiently small and N is sufficiently large. The computation of this formula can be greatly speeded up if we note that

$$e^{-(x'-n'h)^2} = e^{-x'^2} e^{-(n'h)^2} \left(e^{2x'h} \right)^{n'} \quad (6.9.5)$$

The first factor is computed once, the second is an array of constants to be stored, and the third can be computed recursively, so that only two exponentials need be evaluated. Advantage is also taken of the symmetry of the coefficients $e^{-(n'h)^2}$ by breaking up the summation into positive and negative values of n' separately.

In the following routine, the choices $h = 0.4$ and $N = 11$ are made. Because of the symmetry of the summations and the restriction to odd values of n , the limits on the for loops are 0 to 5. The accuracy of the result in this version is about 2×10^{-7} . In order to maintain relative accuracy near $x = 0$, where $F(x)$ vanishes, the program branches to the evaluation of the power series [2] for $F(x)$, for $|x| < 0.2$.

```
Doub dawson(const Doub x) {
Returns Dawson's integral  $F(x) = \exp(-x^2) \int_0^x \exp(t^2) dt$  for any real  $x$ .
    static const Int NMAX=6;
    static VecDoub c(NMAX);
    static Bool init = true;
    static const Doub H=0.4, A1=2.0/3.0, A2=0.4, A3=2.0/7.0;
    Int i,n0;
    Doub d1,d2,e1,e2,sum,x2,xp,xx,ans;
    if (init) {
        init=false;
        for (i=0;i<NMAX;i++) c[i]=exp(-SQR((2.0*i+1.0)*H));
    }
    if (abs(x) < 0.2) {
        Use series expansion.
        x2=x*x;
        ans=x*(1.0-A1*x2*(1.0-A2*x2*(1.0-A3*x2)));
    } else {
        Use sampling theorem representation.
        xx=abs(x);
        n0=2*Int(0.5*xx/H+0.5);
        xp=xx-n0*H;
        e1=exp(2.0*xp*H);
```

dawson.h

```

e2=e1*e1;
d1=n0+1;
d2=d1-2.0;
sum=0.0;
for (i=0;i<NMAX;i++,d1+=2.0,d2-=2.0,e1*=e2)
    sum += c[i]*(e1/d1+1.0/(d2*e1));
ans=0.5641895835*SIGN(exp(-xp*xp),x)*sum;    Constant is 1/√π.
}
return ans;
}

```

Other methods for computing Dawson's integral are also known [2,3].

CITED REFERENCES AND FURTHER READING:

- Rybicki, G.B. 1989, "Dawson's Integral and The Sampling Theorem," *Computers in Physics*, vol. 3, no. 2, pp. 85–87.[1]
 Cody, W.J., Pociorek, K.A., and Thatcher, H.C. 1970, "Chebyshev Approximations for Dawson's Integral," *Mathematics of Computation*, vol. 24, pp. 171–178.[2]
 McCabe, J.H. 1974, "A Continued Fraction Expansion, with a Truncation Error Estimate, for Dawson's Integral," *Mathematics of Computation*, vol. 28, pp. 811–816.[3]

6.10 Generalized Fermi-Dirac Integrals

The generalized Fermi-Dirac integral is defined as

$$F_k(\eta, \theta) = \int_0^\infty \frac{x^k (1 + \frac{1}{2}\theta x)^{1/2}}{e^{x-\eta} + 1} dx \quad (6.10.1)$$

It occurs, for example, in astrophysical applications with θ nonnegative and arbitrary η . In condensed matter physics one usually has the simpler case of $\theta = 0$ and omits the "generalized" from the name of the function. The important values of k are $-1/2$, $1/2$, $3/2$, and $5/2$, but we'll consider arbitrary values greater than -1 . Watch out for an alternative definition that multiplies the integral by $1/\Gamma(k+1)$.

For $\eta \ll -1$ and $\eta \gg 1$ there are useful series expansions for these functions (see, e.g., [1]). These give, for example,

$$\begin{aligned}
 F_{1/2}(\eta, \theta) &\rightarrow \frac{1}{\sqrt{2\theta}} e^\eta e^{1/\theta} K_1\left(\frac{1}{\theta}\right), & \eta \rightarrow -\infty \\
 F_{1/2}(\eta, \theta) &\rightarrow \frac{1}{2\sqrt{2}} \eta^{3/2} \frac{y \sqrt{1+y^2} - \sinh^{-1} y}{(\sqrt{1+y^2} - 1)^{3/2}}, & \eta \rightarrow \infty
 \end{aligned} \quad (6.10.2)$$

Here y is defined by

$$1 + y^2 = (1 + \eta\theta)^2 \quad (6.10.3)$$

It is the middle range of η values that is difficult to handle.

For $\theta = 0$, Macleod [2] has given Chebyshev expansions accurate to 10^{-16} for the four important k values, covering all η values. In this case, one need look no further for an algorithm. Goano [3] handles arbitrary k for $\theta = 0$. For nonzero θ ,

it is reasonable to compute the functions by direct integration, using variable transformation to get rapidly converging quadratures [4]. (Of course, this works also for $\theta = 0$, but is not as efficient.) The usual transformation $x = \exp(t - e^{-t})$ handles the singularity at $x = 0$ and the exponential fall off at large x (cf. equation 4.5.14). For $\eta \gtrsim 15$, it is better to split the integral into two regions, $[0, \eta]$ and $[\eta, \eta + 60]$. (The contribution beyond $\eta + 60$ is negligible.) Each of these integrals can then be done with the DE rule. Between 60 and 500 function evaluations give full double precision, larger η requiring more function evaluations. A more efficient strategy would replace the quadrature by a series expansion for large η .

In the implementation below, note how `operator()` is overloaded to define both a function of one variable (for `Trapzd`) and a function of two variables (for `DERule`). Note also the syntax

```
Trapzd<Fermi> s(*this,a,b);
```

for declaring a `Trapzd` object inside the `Fermi` object itself.

```
struct Fermi {
    Doub kk,etaa,thetaa;
    Doub operator() (const Doub t);
    Doub operator() (const Doub x, const Doub del);
    Doub val(const Doub k, const Doub eta, const Doub theta);
};
```

fermi.h

```
Doub Fermi::operator() (const Doub t) {
    Integrand for trapezoidal quadrature of generalized Fermi-Dirac integral with transformation
    x = exp(t - e^{-t}).
    Doub x;
    x=exp(t-exp(-t));
    return x*(1.0+exp(-t))*pow(x,kk)*sqrt(1.0+thetaa*0.5*x)/
        (exp(x-etaa)+1.0);
}
```

```
Doub Fermi::operator() (const Doub x, const Doub del) {
    Integrand for DE rule quadrature of generalized Fermi-Dirac integral.
    if (x < 1.0)
        return pow(del,kk)*sqrt(1.0+thetaa*0.5*x)/(exp(x-etaa)+1.0);
    else
        return pow(x,kk)*sqrt(1.0+thetaa*0.5*x)/(exp(x-etaa)+1.0);
}
```

```
Doub Fermi::val(const Doub k, const Doub eta, const Doub theta)
    Computes the generalized Fermi-Dirac integral  $F_k(\eta, \theta)$ , where  $k > -1$  and  $\theta \geq 0$ . The
    accuracy is approximately the square of the parameter EPS. NMAX limits the total number of
    quadrature steps.
    {
        const Doub EPS=3.0e-9;
        const Int NMAX=11;
        Doub a,aa,b,bb,hmax,olds,sum;
        kk=k;
        etaa=eta;
        thetaa=theta;
        if (eta <= 15.0) {
            a=-4.5;
            b=5.0;
            Trapzd<Fermi> s(*this,a,b);
            for (Int i=1;i<=NMAX;i++) {
                sum=s.next();
                if (i > 3)
                    if (abs(sum-olds) <= EPS*abs(olds))
```

Load the arguments into the member variables
for use in the function evaluations.

Set limits for $x = \exp(t - e^{-t})$ mapping.

Test for convergence.

```

        return sum;
    olds=sum;                                Save value for next convergence test.
}
}
else {
    a=0.0;                                    Set limits for DE rule.
    b=eta;
    aa=eta;
    bb=eta+60.0;
    hmax=4.3;                                Big enough to handle negative  $k$  or large  $\eta$ .
    DErule<Fermi> s(*this,a,b,hmax);
    DErule<Fermi> ss(*this,aa,bb,hmax);
    for (Int i=1;i<=NMAX;i++) {
        sum=s.next()+ss.next();
        if (i > 3)
            if (abs(sum-olds) <= EPS*abs(olds))
                return sum;
        olds=sum;
    }
}
throw("no convergence in fermi");
return 0.0;
}

```

You get values of the Fermi-Dirac functions by declaring a Fermi object:

```
Fermi ferm;
```

and then making repeated calls to the val function:

```
ans=ferm.val(k,eta,theta);
```

Other quadrature methods exist for these functions [5-7]. A reasonably efficient method [8] involves trapezoidal quadrature with “pole correction,” but it is restricted to $\theta \lesssim 0.2$. Generalized Bose-Einstein integrals can also be computed by the DE rule or the methods in these references.

CITED REFERENCES AND FURTHER READING:

- Cox, J.P., and Giuli, R.T. 1968, *Principles of Stellar Structure* (New York: Gordon and Breach), vol. II, §24.7.[1]
- Macleod, A.J. 1998, “Fermi-Dirac Functions of Order $-1/2$, $1/2$, $3/2$, $5/2$,” *ACM Transactions on Mathematical Software*, vol. 24, pp. 1–12. (Algorithm 779, available from netlib.)[2]
- Goano, M. 1995, “Computation of the Complete and Incomplete Fermi-Dirac Integral,” *ACM Transactions on Mathematical Software*, vol. 21, pp. 221–232. (Algorithm 745, available from netlib.)[3]
- Natarajan, A., and Kumar, N.M. 1993, “On the Numerical Evaluation of the Generalised Fermi-Dirac Integrals,” *Computer Physics Communications*, vol. 76, pp. 48–50.[4]
- Pichon, B. 1989, “Numerical Calculation of the Generalized Fermi-Dirac Integrals,” *Computer Physics Communications*, vol. 55, pp. 127–136.[5]
- Sagar, R.P. 1991, “A Gaussian Quadrature for the Calculation of Generalized Fermi-Dirac Integrals,” *Computer Physics Communications*, vol. 66, pp. 271–275.[6]
- Gautschi, W. 1992, “On the Computation of Generalized Fermi-Dirac and Bose-Einstein Integrals,” *Computer Physics Communications*, vol. 74, pp. 233–238.[7]
- Mohankumar, N., and Natarajan, A. 1996, “A Note on the Evaluation of the Generalized Fermi-Dirac Integral,” *Astrophysical Journal*, vol. 458, pp. 233–235.[8]

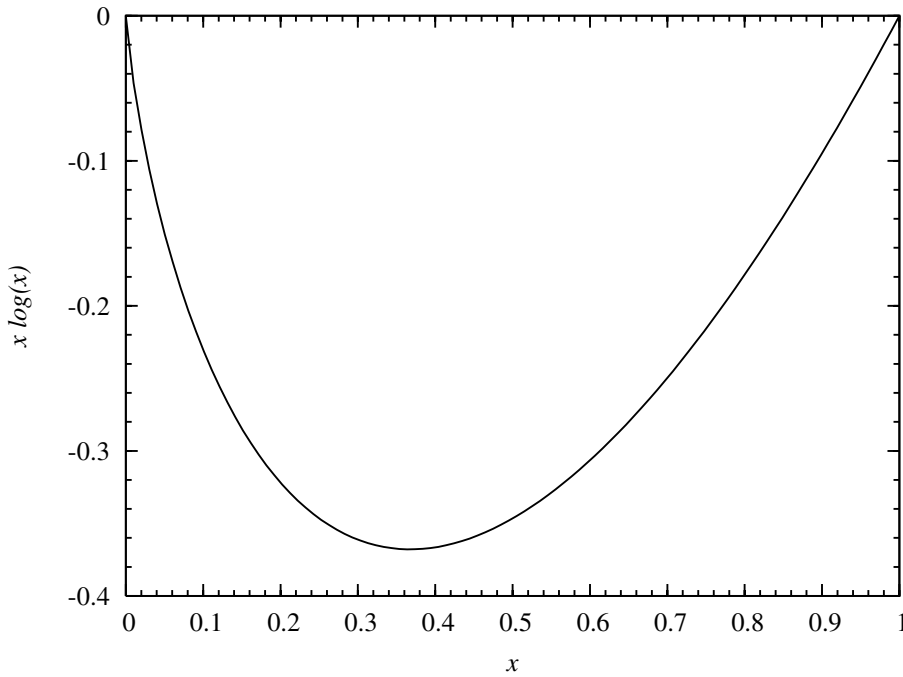


Figure 6.11.1. The function $x \log(x)$ is shown for $0 < x < 1$. Although nearly invisible, an essential singularity at $x = 0$ makes this function tricky to invert.

6.11 Inverse of the Function $x \log(x)$

The function

$$y(x) = x \log(x) \quad (6.11.1)$$

and its inverse function $x(y)$ occur in a number of statistical and information theoretical contexts. Obviously $y(x)$ is nonsingular for all positive x , and easy to evaluate. For x between 0 and 1, it is negative, with a single minimum at $(x, y) = (e^{-1}, -e^{-1})$. The function has the value 0 at $x = 1$, and it has the value 0 as its limit at $x = 0$, since the linear factor x easily dominates the singular logarithm.

Computing the inverse function $x(y)$ is, however, not so easy. (We will need this inverse in §6.14.12.) From the appearance of Figure 6.11.1, it might seem easy to invert the function on its left branch, that is, return a value x between 0 and e^{-1} for every value y between 0 and $-e^{-1}$. However, the lurking logarithmic singularity at $x = 0$ causes difficulties for many methods that you might try.

Polynomial fits work well over any range of y that is less than a decade or so (e.g., from 0.01 to 0.1), but fail badly if you demand high fractional precision extending all the way to $y = 0$.

What about Newton's method? We write

$$\begin{aligned} f(x) &\equiv x \log(x) - y \\ f'(x) &= 1 + \log(x) \end{aligned} \quad (6.11.2)$$

giving the iteration

$$x_{i+1} = x_i - \frac{x_i \log(x_i) - y}{1 + \log(x_i)} \quad (6.11.3)$$

This doesn't work. The problem is not with its rate of convergence, which is of course quadratic for any finite y if we start close enough to the solution (see §9.4). The problem is that the region in which it converges at all is very small, especially as $y \rightarrow 0$. So, if we don't already have a good approximation as we approach the singularity, we are sunk.

If we change variables, we can get different (not computationally equivalent) versions of Newton's method. For example, let

$$u \equiv \log(x), \quad x = e^u \quad (6.11.4)$$

Newton's method in u looks like this:

$$\begin{aligned} f(u) &= ue^u - y \\ f'(u) &= (1 + u)e^u \\ u_{i+1} &= u_i - \frac{u_i - e^{-u_i} y}{1 + u_i} \end{aligned} \quad (6.11.5)$$

But it turns out that iteration (6.11.5) is no better than (6.11.3).

The observation that leads to a good solution is that, since its log term varies only slowly, $y = x \log(x)$ is only very modestly curved *when it is plotted in log-log coordinates*. (Actually it is the negative of y that is plotted, since log-log coordinates require positive quantities.) Algebraically, we rewrite equation (6.11.1) as

$$(-y) = (-u)e^u \quad (6.11.6)$$

(with u as defined above) and take logarithms, giving

$$\log(-y) = u + \log(-u) \quad (6.11.7)$$

This leads to the Newton formulas,

$$\begin{aligned} f(u) &= u + \log(-u) - \log(-y) \\ f'(u) &= \frac{u + 1}{u} \\ u_{i+1} &= u_i + \frac{u_i}{u_i + 1} \left[\log\left(\frac{y}{u_i}\right) - u_i \right] \end{aligned} \quad (6.11.8)$$

It turns out that the iteration (6.11.8) converges quadratically over quite a broad region of initial guesses. For $-0.2 < y < 0$, you can just choose -10 (for example) as a fixed initial guess. When $-0.2 < y < -e^{-1}$, one can use the Taylor series expansion around $x = e^{-1}$,

$$y(x - e^{-1}) = -e^{-1} + \frac{1}{2}e(x - e^{-1})^2 + \dots \quad (6.11.9)$$

which yields

$$x \approx e^{-1} - \sqrt{2e^{-1}(y + e^{-1})} \quad (6.11.10)$$

With these initial guesses, (6.11.8) never takes more than six iterations to converge to double precision accuracy, and there is just one log and a few arithmetic operations per iteration. The implementation looks like this:

```
Doub invxlogx(Doub y) {
```

ksdist.h

For negative y , $0 > y > -e^{-1}$, return x such that $y = x \log(x)$. The value returned is always the smaller of the two roots and is in the range $0 < x < e^{-1}$.

```
    const Doub ooe = 0.367879441171442322;
    Doub t,u,to=0.;
    if (y >= 0. || y <= -ooe) throw("no such inverse value");
    if (y < -0.2) u = log(ooe-sqrt(2*ooe*(y+ooe))); First approximation by inverse
    else u = -10.;                                of Taylor series.
    do {                                           See text for derivation.
        u += (t=(log(y/u)-u)*(u/(1.+u)));
        if (t < 1.e-8 && abs(t+to)<0.01*abs(t)) break;
        to = t;
    } while (abs(t/u) > 1.e-15);
    return exp(u);
}
```

6.12 Elliptic Integrals and Jacobian Elliptic Functions

Elliptic integrals occur in many applications, because any integral of the form

$$\int R(t, s) dt \quad (6.12.1)$$

where R is a rational function of t and s , and s is the square root of a cubic or quartic polynomial in t , can be evaluated in terms of elliptic integrals. Standard references [1] describe how to carry out the reduction, which was originally done by Legendre. Legendre showed that only three basic elliptic integrals are required. The simplest of these is

$$I_1 = \int_y^x \frac{dt}{\sqrt{(a_1 + b_1 t)(a_2 + b_2 t)(a_3 + b_3 t)(a_4 + b_4 t)}} \quad (6.12.2)$$

where we have written the quartic s^2 in factored form. In standard integral tables [2], one of the limits of integration is always a zero of the quartic, while the other limit lies closer than the next zero, so that there is no singularity within the interval. To evaluate I_1 , we simply break the interval $[y, x]$ into subintervals, each of which either begins or ends on a singularity. The tables, therefore, need only distinguish the eight cases in which each of the four zeros (ordered according to size) appears as the upper or lower limit of integration. In addition, when one of the b 's in (6.12.2) tends to zero, the quartic reduces to a cubic, with the largest or smallest singularity moving to $\pm\infty$; this leads to eight more cases (actually just special cases of the first eight). The 16 cases in total are then usually tabulated in terms of Legendre's standard elliptic integral of the first kind, which we will define below. By a change of the variable of integration t , the zeros of the quartic are mapped to standard locations on the real axis. Then only two dimensionless parameters are needed to tabulate Legendre's integral. However, the symmetry of the original integral (6.12.2) under permutation of the roots is concealed in Legendre's notation. We will get back to Legendre's notation below. But first, here is a better approach:

Carlson [3] has given a new definition of a standard elliptic integral of the first kind,

$$R_F(x, y, z) = \frac{1}{2} \int_0^\infty \frac{dt}{\sqrt{(t+x)(t+y)(t+z)}} \quad (6.12.3)$$

where x , y , and z are nonnegative and at most one is zero. By standardizing the range of integration, he retains permutation symmetry for the zeros. (Weierstrass' canonical form also has this property.) Carlson first shows that when x or y is a zero of the quartic in (6.12.2), the integral I_1 can be written in terms of R_F in a form that is symmetric under permutation of the remaining three zeros. In the general case, when neither x nor y is a zero, two such R_F functions can be combined into a single one by an *addition theorem*, leading to the fundamental formula

$$I_1 = 2R_F(U_{12}^2, U_{13}^2, U_{14}^2) \quad (6.12.4)$$

where

$$U_{ij} = (X_i X_j Y_k Y_m + Y_i Y_j X_k X_m)/(x - y) \quad (6.12.5)$$

$$X_i = (a_i + b_i x)^{1/2}, \quad Y_i = (a_i + b_i y)^{1/2} \quad (6.12.6)$$

and i, j, k, m is any permutation of 1, 2, 3, 4. A short-cut in evaluating these expressions is

$$\begin{aligned} U_{13}^2 &= U_{12}^2 - (a_1 b_4 - a_4 b_1)(a_2 b_3 - a_3 b_2) \\ U_{14}^2 &= U_{12}^2 - (a_1 b_3 - a_3 b_1)(a_2 b_4 - a_4 b_2) \end{aligned} \quad (6.12.7)$$

The U 's correspond to the three ways of pairing the four zeros, and I_1 is thus manifestly symmetric under permutation of the zeros. Equation (6.12.4) therefore reproduces all 16 cases when one limit is a zero, and also includes the cases when neither limit is a zero.

Thus Carlson's function allows arbitrary ranges of integration and arbitrary positions of the branch points of the integrand relative to the interval of integration. To handle elliptic integrals of the second and third kinds, Carlson defines the standard integral of the third kind as

$$R_J(x, y, z, p) = \frac{3}{2} \int_0^\infty \frac{dt}{(t+p)\sqrt{(t+x)(t+y)(t+z)}} \quad (6.12.8)$$

which is symmetric in x , y , and z . The degenerate case when two arguments are equal is denoted

$$R_D(x, y, z) = R_J(x, y, z, z) \quad (6.12.9)$$

and is symmetric in x and y . The function R_D replaces Legendre's integral of the second kind. The degenerate form of R_F is denoted

$$R_C(x, y) = R_F(x, y, y) \quad (6.12.10)$$

It embraces logarithmic, inverse circular, and inverse hyperbolic functions.

Carlson [4-7] gives integral tables in terms of the exponents of the linear factors of the quartic in (6.12.1). For example, the integral where the exponents are $(\frac{1}{2}, \frac{1}{2}, -\frac{1}{2}, -\frac{3}{2})$ can be expressed as a single integral in terms of R_D ; it accounts for 144 separate cases in Gradshteyn and Ryzhik [2]!

Refer to Carlson's papers [3-8] for some of the practical details in reducing elliptic integrals to his standard forms, such as handling complex-conjugate zeros.

Turn now to the numerical evaluation of elliptic integrals. The traditional methods [9] are Gauss or Landen transformations. *Descending* transformations decrease the modulus k of the Legendre integrals toward zero, and *increasing* transformations increase it toward unity. In these limits the functions have simple analytic expressions. While these methods converge quadratically and are quite satisfactory for integrals of the first and second kinds, they generally lead to loss of significant figures in certain regimes for integrals of the third kind. Carlson's algorithms [10, 11], by contrast, provide a unified method for all three kinds with no significant cancellations.

The key ingredient in these algorithms is the *duplication theorem*:

$$\begin{aligned} R_F(x, y, z) &= 2R_F(x + \lambda, y + \lambda, z + \lambda) \\ &= R_F\left(\frac{x + \lambda}{4}, \frac{y + \lambda}{4}, \frac{z + \lambda}{4}\right) \end{aligned} \quad (6.12.11)$$

where

$$\lambda = (xy)^{1/2} + (xz)^{1/2} + (yz)^{1/2} \quad (6.12.12)$$

This theorem can be proved by a simple change of variable of integration [12]. Equation (6.12.11) is iterated until the arguments of R_F are nearly equal. For equal arguments we have

$$R_F(x, x, x) = x^{-1/2} \quad (6.12.13)$$

When the arguments are close enough, the function is evaluated from a fixed Taylor expansion about (6.12.13) through fifth-order terms. While the iterative part of the algorithm is only linearly convergent, the error ultimately decreases by a factor of $4^6 = 4096$ for each iteration. Typically only two or three iterations are required, perhaps six or seven if the initial values of the arguments have huge ratios. We list the algorithm for R_F here, and refer you to Carlson's paper [10] for the other cases.

Stage 1: For $n = 0, 1, 2, \dots$ compute

$$\begin{aligned} \mu_n &= (x_n + y_n + z_n)/3 \\ X_n &= 1 - (x_n/\mu_n), \quad Y_n = 1 - (y_n/\mu_n), \quad Z_n = 1 - (z_n/\mu_n) \\ \epsilon_n &= \max(|X_n|, |Y_n|, |Z_n|) \end{aligned}$$

If $\epsilon_n < \text{tol}$, go to Stage 2; else compute

$$\begin{aligned} \lambda_n &= (x_n y_n)^{1/2} + (x_n z_n)^{1/2} + (y_n z_n)^{1/2} \\ x_{n+1} &= (x_n + \lambda_n)/4, \quad y_{n+1} = (y_n + \lambda_n)/4, \quad z_{n+1} = (z_n + \lambda_n)/4 \end{aligned}$$

and repeat this stage.

Stage 2: Compute

$$\begin{aligned} E_2 &= X_n Y_n - Z_n^2, \quad E_3 = X_n Y_n Z_n \\ R_F &= (1 - \frac{1}{10} E_2 + \frac{1}{14} E_3 + \frac{1}{24} E_2^2 - \frac{3}{44} E_2 E_3)/(\mu_n)^{1/2} \end{aligned}$$

In some applications the argument p in R_J or the argument y in R_C is negative, and the Cauchy principal value of the integral is required. This is easily handled by using the formulas

$$\begin{aligned} R_J(x, y, z, p) &= \\ &[(\gamma - y)R_J(x, y, z, \gamma) - 3R_F(x, y, z) + 3R_C(xz/y, p\gamma/y)]/(\gamma - p) \end{aligned} \quad (6.12.14)$$

where

$$\gamma \equiv y + \frac{(z - y)(y - x)}{y - p} \quad (6.12.15)$$

is positive if p is negative, and

$$R_C(x, y) = \left(\frac{x}{x - y}\right)^{1/2} R_C(x - y, -y) \quad (6.12.16)$$

The Cauchy principal value of R_J has a zero at some value of $p < 0$, so (6.12.14) will give some loss of significant figures near the zero.

elliptint.h

Doub rf(const Doub x, const Doub y, const Doub z) {
 Computes Carlson's elliptic integral of the first kind, $R_F(x, y, z)$. x , y , and z must be non-negative, and at most one can be zero.

```
    static const Doub ERRTOL=0.0025, THIRD=1.0/3.0, C1=1.0/24.0, C2=0.1,
        C3=3.0/44.0, C4=1.0/14.0;
    static const Doub TINY=5.0*numeric_limits<Doub>::min(),
        BIG=0.2*numeric_limits<Doub>::max();
    Doub alamb, ave, delx, dely, delz, e2, e3, sqrtx, sqrtz, sqrtz, xt, yt, zt;
    if (MIN(MIN(x,y),z) < 0.0 || MIN(MIN(x+y,x+z),y+z) < TINY ||
        MAX(MAX(x,y),z) > BIG) throw("invalid arguments in rf");
    xt=x;
    yt=y;
    zt=z;
    do {
        sqrtx=sqrt(xt);
        sqrtz=sqrt(yt);
        sqrtz=sqrt(zt);
        alamb=sqrtx*(sqrtz+sqrtz)+sqrtz*sqrtz;
        xt=0.25*(xt+alamb);
        yt=0.25*(yt+alamb);
        zt=0.25*(zt+alamb);
        ave=THIRD*(xt+yt+zt);
        delx=(ave-xt)/ave;
        dely=(ave-yt)/ave;
        delz=(ave-zt)/ave;
    } while (MAX(MAX(abs(delx),abs(dely)),abs(delz)) > ERRTOL);
    e2=delx*dely-delz*delz;
    e3=delx*dely*delz;
    return (1.0+(C1*e2-C2-C3*e3)*e2+C4*e3)/sqrt(ave);
}
```

A value of 0.0025 for the error tolerance parameter gives full double precision (16 significant digits). Since the error scales as ϵ_n^6 , we see that 0.08 would be adequate for single precision (7 significant digits), but would save at most two or three more iterations. Since the coefficients of the sixth-order truncation error are different for the other elliptic functions, these values for the error tolerance should be set to 0.04 (single precision) or 0.0012 (double precision) in the algorithm for R_C , and 0.05 or 0.0015 for R_J and R_D . As well as being an algorithm in its own right for certain combinations of elementary functions, the algorithm for R_C is used repeatedly in the computation of R_J .

The C++ implementations test the input arguments against two machine-dependent constants, TINY and BIG, to ensure that there will be no underflow or overflow during the computation. You can always extend the range of admissible argument values by using the homogeneity relations (6.12.22), below.

elliptint.h

Doub rd(const Doub x, const Doub y, const Doub z) {
 Computes Carlson's elliptic integral of the second kind, $R_D(x, y, z)$. x and y must be nonnegative, and at most one can be zero. z must be positive.

```
    static const Doub ERRTOL=0.0015, C1=3.0/14.0, C2=1.0/6.0, C3=9.0/22.0,
        C4=3.0/26.0, C5=0.25*C3, C6=1.5*C4;
    static const Doub TINY=2.0*pow(numeric_limits<Doub>::max(),-2./3.),
        BIG=0.1*ERRTOL*pow(numeric_limits<Doub>::min(),-2./3.);
    Doub alamb, ave, delx, dely, delz, ea, eb, ec, ed, ee, fac, sqrtx, sqrtz,
        sqrtz, sum, xt, yt, zt;
    if (MIN(x,y) < 0.0 || MIN(x+y,z) < TINY || MAX(MAX(x,y),z) > BIG)
        throw("invalid arguments in rd");
    xt=x;
    yt=y;
    zt=z;
    sum=0.0;
    fac=1.0;
    do {
        sqrtx=sqrt(xt);
```

```

    sqrty=sqrt(yt);
    sqrtz=sqrt(zt);
    alamb=sqrtx*(sqrty+sqrtz)+sqrty*sqrtz;
    sum += fac/(sqrtz*(zt+alamb));
    fac=0.25*fac;
    xt=0.25*(xt+alamb);
    yt=0.25*(yt+alamb);
    zt=0.25*(zt+alamb);
    ave=0.2*(xt+yt+3.0*zt);
    delx=(ave-xt)/ave;
    dely=(ave-yt)/ave;
    delz=(ave-zt)/ave;
} while (MAX(MAX(abs(delx),abs(dely)),abs(delz)) > ERRTOL);
ea=delx*dely;
eb=delz*delz;
ec=ea-eb;
ed=ea-6.0*eb;
ee=ed+ec+ec;
return 3.0*sum+fac*(1.0+ed*(-C1+C5*ed-C6*delz*ee)
    +delz*(C2*ee+delz*(-C3*ec+delz*C4*ea)))/(ave*sqrt(ave));
}

```

Doub rj(const Doub x, const Doub y, const Doub z, const Doub p) { elliptint.h
 Computes Carlson's elliptic integral of the third kind, $R_J(x, y, z, p)$. x , y , and z must be nonnegative, and at most one can be zero. p must be nonzero. If $p < 0$, the Cauchy principal value is returned.

```

    static const Doub ERRTOL=0.0015, C1=3.0/14.0, C2=1.0/3.0, C3=3.0/22.0,
        C4=3.0/26.0, C5=0.75*C3, C6=1.5*C4, C7=0.5*C2, C8=C3+C3;
    static const Doub TINY=pow(5.0*numeric_limits<Doub>::min(),1./3.),
        BIG=0.3*pow(0.2*numeric_limits<Doub>::max(),1./3.);
    Doub a,alamb,alpha,ans,ave,b,beta,delp,dex,dely,delz,ea,eb,ec,ed,ee,
        fac,pt,rcx,rho,sqrtx,sqrty,sqrtz,sum,tau,xt,yt,zt;
    if (MIN(MIN(x,y),z) < 0.0 || MIN(MIN(x+y,x+z),MIN(y+z,abs(p))) < TINY
        || MAX(MAX(x,y),MAX(z,abs(p))) > BIG) throw("invalid arguments in rj");
    sum=0.0;
    fac=1.0;
    if (p > 0.0) {
        xt=x;
        yt=y;
        zt=z;
        pt=p;
    } else {
        xt=MIN(MIN(x,y),z);
        zt=MAX(MAX(x,y),z);
        yt=x+y+z-xt-zt;
        a=1.0/(yt-p);
        b=a*(zt-yt)*(yt-xt);
        pt=yt+b;
        rho=xt*zt/yt;
        tau=p*pt/yt;
        rcx=rc(rho,tau);
    }
    do {
        sqrtx=sqrt(xt);
        sqrty=sqrt(yt);
        sqrtz=sqrt(zt);
        alamb=sqrtx*(sqrty+sqrtz)+sqrty*sqrtz;
        alpha=SQR(pt*(sqrtx+sqrty+sqrtz)+sqrtx*sqrty*sqrtz);
        beta=pt*SQR(pt+alamb);
        sum += fac*rc(alpha,beta);
        fac=0.25*fac;
        xt=0.25*(xt+alamb);
        yt=0.25*(yt+alamb);
    }

```

```

    zt=0.25*(zt+alamb);
    pt=0.25*(pt+alamb);
    ave=0.2*(xt+yt+zt+pt+pt);
    delx=(ave-xt)/ave;
    dely=(ave-yt)/ave;
    delz=(ave-zt)/ave;
    delp=(ave-pt)/ave;
} while (MAX(MAX(abs(delx),abs(dely)),
    MAX(abs(delz),abs(delp))) > ERRTOL);
ea=delx*(dely+delz)+dely*delz;
eb=delx*dely*delz;
ec=delp*delp;
ed=ea-3.0*ec;
ee=eb+2.0*delp*(ea-ec);
ans=3.0*sum+fac*(1.0+ed*(-C1+C5*ed-C6*ee)+eb*(C7+delp*(-C8+delp*C4))
    +delp*ea*(C2-delp*C3)-C2*delp*ec)/(ave*sqrt(ave));
if (p <= 0.0) ans=a*(b*ans+3.0*(rcx-rf(xt,yt,zt)));
return ans;
}

```

elliptint.h

```

Doubl rc(const Doubl x, const Doubl y) {
    Computes Carlson's degenerate elliptic integral,  $R_C(x, y)$ .  $x$  must be nonnegative and  $y$  must
    be nonzero. If  $y < 0$ , the Cauchy principal value is returned.
    static const Doubl ERRTOL=0.0012, THIRD=1.0/3.0, C1=0.3, C2=1.0/7.0,
        C3=0.375, C4=9.0/22.0;
    static const Doubl TINY=5.0*numeric_limits<Doubl>::min(),
        BIG=0.2*numeric_limits<Doubl>::max(), COMP1=2.236/sqrt(TINY),
        COMP2=SQR(TINY*BIG)/25.0;
    Doubl alamb,ave,s,w,xt,yt;
    if (x < 0.0 || y == 0.0 || (x+abs(y)) < TINY || (x+abs(y)) > BIG ||
        (y<-COMP1 && x > 0.0 && x < COMP2)) throw("invalid arguments in rc");
    if (y > 0.0) {
        xt=x;
        yt=y;
        w=1.0;
    } else {
        xt=x-y;
        yt= -y;
        w=sqrt(x)/sqrt(xt);
    }
    do {
        alamb=2.0*sqrt(xt)*sqrt(yt)+yt;
        xt=0.25*(xt+alamb);
        yt=0.25*(yt+alamb);
        ave=THIRD*(xt+yt+yt);
        s=(yt-ave)/ave;
    } while (abs(s) > ERRTOL);
    return w*(1.0+s*s*(C1+s*(C2+s*(C3+s*C4)))/sqrt(ave);
}

```

At times you may want to express your answer in Legendre's notation. Alternatively, you may be given results in that notation and need to compute their values with the programs given above. It is a simple matter to transform back and forth. The *Legendre elliptic integral of the first kind* is defined as

$$F(\phi, k) \equiv \int_0^\phi \frac{d\theta}{\sqrt{1 - k^2 \sin^2 \theta}} \quad (6.12.17)$$

The *complete elliptic integral of the first kind* is given by

$$K(k) \equiv F(\pi/2, k) \quad (6.12.18)$$

In terms of R_F ,

$$\begin{aligned} F(\phi, k) &= \sin \phi R_F(\cos^2 \phi, 1 - k^2 \sin^2 \phi, 1) \\ K(k) &= R_F(0, 1 - k^2, 1) \end{aligned} \quad (6.12.19)$$

The Legendre elliptic integral of the second kind and the complete elliptic integral of the second kind are given by

$$\begin{aligned} E(\phi, k) &\equiv \int_0^\phi \sqrt{1 - k^2 \sin^2 \theta} d\theta \\ &= \sin \phi R_F(\cos^2 \phi, 1 - k^2 \sin^2 \phi, 1) \\ &\quad - \frac{1}{3} k^2 \sin^3 \phi R_D(\cos^2 \phi, 1 - k^2 \sin^2 \phi, 1) \\ E(k) &\equiv E(\pi/2, k) = R_F(0, 1 - k^2, 1) - \frac{1}{3} k^2 R_D(0, 1 - k^2, 1) \end{aligned} \quad (6.12.20)$$

Finally, the Legendre elliptic integral of the third kind is

$$\begin{aligned} \Pi(\phi, n, k) &\equiv \int_0^\phi \frac{d\theta}{(1 + n \sin^2 \theta) \sqrt{1 - k^2 \sin^2 \theta}} \\ &= \sin \phi R_F(\cos^2 \phi, 1 - k^2 \sin^2 \phi, 1) \\ &\quad - \frac{1}{3} n \sin^3 \phi R_J(\cos^2 \phi, 1 - k^2 \sin^2 \phi, 1, 1 + n \sin^2 \phi) \end{aligned} \quad (6.12.21)$$

(Note that this sign convention for n is opposite that of Abramowitz and Stegun [13], and that their $\sin \alpha$ is our k .)

```
Doub ellf(const Doub phi, const Doub ak) {
Legendre elliptic integral of the first kind  $F(\phi, k)$ , evaluated using Carlson's function  $R_F$ . The
argument ranges are  $0 \leq \phi \leq \pi/2$ ,  $0 \leq k \sin \phi \leq 1$ .
    Doub s=sin(phi);
    return s*rf(SQR(cos(phi)), (1.0-s*ak)*(1.0+s*ak), 1.0);
}
```

elliptint.h

```
Doub elle(const Doub phi, const Doub ak) {
Legendre elliptic integral of the second kind  $E(\phi, k)$ , evaluated using Carlson's functions  $R_D$ 
and  $R_F$ . The argument ranges are  $0 \leq \phi \leq \pi/2$ ,  $0 \leq k \sin \phi \leq 1$ .
    Doub cc,q,s;
    s=sin(phi);
    cc=SQR(cos(phi));
    q=(1.0-s*ak)*(1.0+s*ak);
    return s*(rf(cc,q,1.0)-(SQR(s*ak))*rd(cc,q,1.0)/3.0);
}
```

elliptint.h

```
Doub ellpi(const Doub phi, const Doub en, const Doub ak) {
Legendre elliptic integral of the third kind  $\Pi(\phi, n, k)$ , evaluated using Carlson's functions  $R_J$ 
and  $R_F$ . (Note that the sign convention on  $n$  is opposite that of Abramowitz and Stegun.)
The ranges of  $\phi$  and  $k$  are  $0 \leq \phi \leq \pi/2$ ,  $0 \leq k \sin \phi \leq 1$ .
    Doub cc,enss,q,s;
    s=sin(phi);
    enss=en*s*s;
    cc=SQR(cos(phi));
    q=(1.0-s*ak)*(1.0+s*ak);
    return s*(rf(cc,q,1.0)-enss*rj(cc,q,1.0,1.0+enss)/3.0);
}
```

elliptint.h